

# Projet Data E-commerce : Guide d'Implémentation Complet

## 1. Architecture et Sources de Données

Pour répondre à l'exigence des sources multiples, l'architecture de ce projet s'appuiera sur deux canaux de données distincts qui seront croisés et analysés. Le but est de simuler un environnement d'entreprise réaliste où les informations proviennent de différents systèmes.

- **Source 1 : L'API de la boutique (ex: API PrestaShop).** Cette source fournira les données transactionnelles en temps réel (les commandes, les clients, les détails des produits).
- **Source 2 : Fichier plat (CSV ou Excel) ou API externe.** Cette source fournira des données complémentaires, comme un fichier d'avis clients ou un référentiel de coûts logistiques par pays.

## 2. Les Pré-requis Techniques

Avant de commencer à coder, il est indispensable de préparer son environnement de travail. Le tableau ci-dessous récapitule les outils nécessaires pour chaque composante du projet.

| Composant du Projet | Outil / Technologie Recommandée | Description & Installation                                                                                                                                                          |
|---------------------|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Langage & ETL       | Python (avec Pandas, Requests)  | Python est le standard pour l'ingénierie des données. La bibliothèque 'requests' permet d'interroger les API, et 'pandas' permet de manipuler les DataFrames (tableaux de données). |
| Orchestrateur       | Apache Airflow                  | Il s'installe idéalement sur un environnement Linux. L'utilisation de WSL (Windows Subsystem for Linux) est parfaite pour faire                                                     |

| Composant du Projet      | Outil / Technologie Recommandée | Description & Installation                                                                                          |
|--------------------------|---------------------------------|---------------------------------------------------------------------------------------------------------------------|
|                          |                                 | tourner Airflow localement sur une machine Windows.                                                                 |
| <b>Data Warehouse</b>    | PostgreSQL                      | Une base de données relationnelle robuste, très bien gérée par les environnements locaux et supportée par Power BI. |
| <b>Datavisualisation</b> | Microsoft Power BI Desktop      | Logiciel gratuit à télécharger pour créer les tableaux de bord et se connecter directement à la base PostgreSQL.    |

### 3. Todo-List Détaillée du Projet

Voici les étapes séquentielles pour mener à bien ce projet de bout en bout.

#### Phase 1 : Mise en place de l'environnement (Semaine 1)

1. **Configurer l'environnement Python** : Créer un environnement virtuel (venv) et installer les bibliothèques requises : pip install pandas requests sqlalchemy psycopg2-binary.
2. **Installer et configurer PostgreSQL** : Créer une base de données nommée dwh\_ecommerce. Créer les tables cibles (par exemple : dim\_clients, dim\_produits, fait\_commandes).
3. **Préparer Airflow sous WSL** : Installer Apache Airflow dans le sous-système Linux, initialiser la base de données interne d'Airflow et créer un utilisateur administrateur.
4. **Sécuriser les accès** : Récupérer la clé API de la boutique (ex: clé webservice PrestaShop) et préparer le fichier CSV des données complémentaires (ex: fichier contenant le taux de satisfaction par produit).

#### Phase 2 : Développement du script ETL (Semaine 2)

### 1. L'Extraction (E) :

- Écrire une fonction Python qui fait un appel GET (via `requests`) à l'API pour récupérer les ventes de la journée.
- Écrire une fonction Python qui lit le fichier CSV des avis clients avec `pandas.read\_csv()`.

### 2. La Transformation (T) :

- Utiliser Pandas pour fusionner (merge) les données de l'API et les données du CSV grâce à un identifiant commun (ex: `product\_id`).
- Nettoyer les données : gérer les valeurs nulles, convertir les dates dans un format standard (YYYY-MM-DD), calculer le revenu total par ligne (quantité \* prix unitaire).

### 3. Le Chargement (L) :

- Utiliser SQLAlchemy pour établir la connexion avec PostgreSQL.
- Insérer le DataFrame propre dans les tables de la base de données dwh\_ecommerce avec la fonction `to\_sql()`.

## Phase 3 : Automatisation avec Airflow (Semaine 3)

1. Créer un fichier DAG (Directed Acyclic Graph) dans le dossier ~/airflow/dags/.
2. Définir la fréquence d'exécution (le "schedule\_interval") sur une exécution quotidienne (ex: @daily).
3. Utiliser le PythonOperator pour exécuter les fonctions d'extraction, de transformation et de chargement créées dans la Phase 2.
4. Configurer les dépendances entre les tâches. Exemple de code de structure :

```
# Structure classique d'un DAG
extract_api_task >> extract_csv_task >> transform_task >> load_db_task
```

## Phase 4 : Datavisualisation avec Power BI (Semaine 4)

1. Ouvrir Power BI Desktop et cliquer sur "Obtenir les données".
2. Choisir PostgreSQL, entrer les identifiants de la base de données locale (localhost) et importer les données nettoyées.
3. Modélisation : Vérifier que les relations entre la table des commandes et les tables des produits/clients sont correctes dans l'onglet Modèle.
4. Créer le rapport final avec les visuels suivants :
  - Une "Carte" (Card) pour le Chiffre d'Affaires total.
  - Un graphique en courbes pour l'évolution quotidienne du chiffre d'affaires.
  - Un graphique en barres combinant le chiffre d'affaires d'un produit avec sa note de

satisfaction moyenne (donnée issue du CSV).